

Data Warehouse & Business Intelligence Lab

Instructor: Dr. Khurram Shahzad

DSF24

Lab 03 | 20-09-2026

Objective: The purpose of this lab is to provide students with practical experience in implementing database triggers and to introduce them to the fundamentals of data movement using Python. Students will learn how to load and manage different types of files, such as Excel, JSON, and CSV, into SQL databases, enabling them to understand the integration of programming with database operations.

Case Study: Why Crescent Textile Mills Needs Triggers for Data Accuracy

Background

Crescent Textile Mills is one of Pakistan's leading textile manufacturers, producing fabrics, garments, and home textiles for both local and international markets. The company manages multiple processes such as sales, production, and inventory; that generate thousands of records every month. Ensuring data accuracy in such a large-scale operation is critical for smooth business performance.

With increasing automation, Crescent has moved to database-driven systems to record transactions. However, relying only on manual checks for correctness (such as valid price entries, stock updates, or employee records) is risky and time-consuming. This is where **database triggers** can help maintain accuracy and consistency automatically.

The Challenge: Human Errors in Data Entry

Despite having digital systems, data entry mistakes occur frequently:

- Sales staff may insert invalid discounts or prices.
- Inventory updates may forget to adjust stock properly after sales.
- Records of deleted or updated entries may get lost without proper logging.

Such issues cause:

- Incorrect financial reports,
- Inventory mismatches, and
- Poor traceability of past changes.

Data Warehouse & Business Intelligence Lab

Instructor: Dr. Khurram Shahzad

DSF24

Real-World Example: Pricing Errors in Local Sales

During seasonal sales in Lahore:

- A staff member mistakenly entered a negative price for a discounted bedsheet set.
- Another order showed an unusually high discount percentage because no validation was in place.
- Inventory stock was reduced incorrectly when multiple updates happened on the same product.

These mistakes required manual correction later, wasting time and delaying shipment reports. If triggers had been in place, they could have prevented such errors automatically.

Consequences of Not Having Triggers

1. **Financial Losses** → Incorrect sales amounts recorded.
2. **Inventory Mismanagement** → Stock not updated properly, leading to mismatches.
3. **Lack of Accountability** → No automatic logging of who changed or deleted a record.
4. **Customer Dissatisfaction** → Delayed shipments and billing mistakes.

The Solution: Database Triggers for Data Accuracy

By implementing **triggers** in Crescent Textile's sales and inventory systems:

- Invalid data entries (e.g., negative prices, empty product names) can be blocked before insertion.
- Automatic logging of changes (before updates or deletes) ensures accountability.
- Stock quantities can be adjusted automatically after sales transactions.

This reduces errors, saves employee effort, and keeps data trustworthy for business intelligence reporting.

Key Lesson for Students

Data Warehouse & Business Intelligence Lab

Instructor: Dr. Khurram Shahzad

DSF24

Triggers are small pieces of database code that run automatically in response to insert, update, or delete operations. Crescent's case shows how **basic triggers** can enforce rules, prevent errors, and maintain logs without depending on manual checks. This is a crucial skill in both **database administration** and **business intelligence**, where data accuracy is the foundation of decision-making.

Lab Tasks

Using the Sales and Inventory tables of Crescent Textile Mills, perform the following trigger-related tasks:

1. Before Insert Trigger – Validate Data

- When inserting into Sales, if the SalePrice is less than or equal to 0, automatically set it to a default value of **1000**.

2. After Insert Trigger – Update Inventory

- After a new sale is recorded, automatically decrease the StockQuantity in the Inventory table for the sold product.

3. Before Update Trigger – Track Changes

- Before updating a record in Sales, insert the old values into a log table Sales_Log to maintain history.

4. Before Delete Trigger – Prevent Mistakes

- Prevent deletion of a sales record if its amount is greater than **50,000** (to avoid accidental removal of high-value sales).

5. After Update Trigger – Maintain Consistency

- If the SalePrice of a product is updated in Sales, automatically update a field LastUpdated (timestamp) in the same row to record when the change happened.

6. Before Insert Trigger – Avoid Empty Names

- If a user tries to insert a NULL or empty value in ProductName, replace it with 'Unknown Product'.

7. After Delete Trigger – Log Deletions

Data Warehouse & Business Intelligence Lab

Instructor: Dr. Khurram Shahzad

DSF24

- After a record is deleted from Sales, insert its SaleID and ProductName into a new table Deleted_Sales for tracking purposes.

Case Study: Why Imtiaz Needs Data Integration for Business Intelligence

Background

Imtiaz Super Market is one of Pakistan's largest retail chains, offering a wide range of products including groceries, household items, clothing, and electronics. With multiple branches across major cities, the supermarket handles thousands of daily transactions from both in-store and online customers. These transactions, along with finance, HR, and inventory data, are stored in different formats and systems.

The Challenge: Fragmented Data

Although Imtiaz collects vast amounts of data, it faces fragmentation and inconsistency:

- Sales transactions are stored in CSV files branch-wise.
- Finance records are maintained separately in Excel sheets.
- HR maintains employee details and attendance in flat files (text/CSV).
- Marketing and supplier data are often exchanged in JSON format.
- Inventory updates are manual and inconsistent across branches.

Because of this, management struggles to get a unified view of operations. Reports require manual data collection from multiple formats, often taking days and delaying decisions.

Real-World Example: Ramadan Rush

During Ramadan, demand for groceries and beverages surges:

- In Karachi, certain cooking oils and soft drinks sold out within days, disappointing customers.

Data Warehouse & Business Intelligence Lab

Instructor: Dr. Khurram Shahzad

DSF24

- In Lahore, the same products were oversupplied, sitting unsold until discounted after Eid.
- Finance struggled to reconcile revenues and expenses quickly due to scattered Excel sheets.
- HR could not correlate staff attendance with sales spikes, missing opportunities to reward high-performing employees.
- Marketing teams could not run timely promotions because customer purchase behavior data was fragmented.

By the time consolidated reports were prepared, the season was already over — and valuable opportunities were lost.

Consequences of Not Having Data Integration

1. Lost Revenue → Stockouts in some cities while other branches carried excess stock.
2. Excess Inventory → Overstock led to waste and heavy discounting.
3. Delayed Reporting → Weeks-long delays in finance and sales reporting.
4. Missed Insights → No link between HR performance, marketing efforts, and sales outcomes.
5. Customer Dissatisfaction → Customers lost trust in Imtiaz's ability to meet seasonal demand.

The Solution: Data Integration & Data Warehouse

A centralized Data Warehouse with integrated ETL (Extract, Transform, Load) processes would:

- Combine sales, finance, HR, marketing, and inventory data from different file formats (CSV, Excel, TXT, JSON).
- Provide a single source of truth for all business operations.
- Enable real-time monitoring of stock levels, staff performance, and branch-wise profitability.
- Support predictive analytics for demand forecasting during peak seasons like Ramadan.
- Empower executives with faster, fact-based decision-making.

Data Warehouse & Business Intelligence Lab

Instructor: Dr. Khurram Shahzad

DSF24

Key Lesson for Students

Imtiaz's case shows that having data alone is not enough. Without **data integration**, organizations remain data-rich but insight-poor. Integrating multiple file formats into a Data Warehouse transforms scattered data into actionable intelligence, helping businesses like Imtiaz serve customers better, boost revenues, and stay ahead of competitors.

Lab Tasks

1. Create Database

- Create a new database named **imtiaz_supermarket** where all four tables will be stored.

2. Sales Data (CSV → Sales Table)

- Load branch-wise sales transaction data from CSV files into the Sales table.

3. Employee Data (CSV → Employee Table)

- Load employee details and attendance data from CSV files into the Employee table.

4. Finance Data (Excel → Finance Table)

- Load finance records from Excel sheets into the Finance table.

5. Inventory Data (JSON → Inventory Table)

- Load supplier and inventory details from JSON files into the Inventory table.